Ders 7 Genişletilmiş Notu: Refactoring'e Giriş, JUnit ile Birim Sınaması ve İlk Kod Kusurları

Ders 7 Genişletilmiş Notu: Refactoring’e Giriş, JUnit ile Birim Sınaması ve İlk Kod Kusurları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Refactoring Nedir?
- 2. Refactoring Nasıl ve Ne Zaman Yapılır?
- 3. Code Smell (Kod Kusuru)
- 4. JUnit ile Birim Sınaması
- 5. Fowler’ın Video Kaset Kiralama Örneği
- 6. Örnek Testler ve Çalıştırılması
- 7. Duplicated Code ve Extract Method
- 8. Long Method
- 9. Yinelenen Kod İçin Diğer Çözümler: Pull Up Method ve Form Template Method
- 10. Long Method İçin Diğer Çözümler ve Decompose Conditional
- 11. Long Parameter List: Introduce Parameter Object ve Preserve Whole Object
- 12. Replace Temp with Query
- 13. Replace Method with Method Object
- 14. Örnek Sorular

1. Refactoring Nedir?

Önceki derslerde tasarım kalıpları işlenmişti. Bu ders, dönemin ikinci büyük konusu olan kod kusurları (code smells) ve refactoring’e geçiş dersidir.

Refactoring genelde yazılımın bakım aşamasında yapılan bir iş olarak bilinir. Ama aslında her aşamada yapılabilir: tasarım da refactor edilir, kod da refactor edilir. Hocanın tanımı şudur:

> Bir kod parçasının **ne yaptığını değiştirmeden nasıl yaptığını** değiştirmeye refactoring denir.

Aynı şeyin başka bir söylenişi: bir birimin dışarıya sunduğu arayüz değişmeden iç yapısının değiştirilmesi. İki tanım aynı kapıya çıkar. Bu tanım dersin ilerleyen kısmında önemli bir ölçüt olacaktır; çünkü bazı refactoring eylemleri metot imzasını değiştirir ve bu, tanıma karşı bir zayıflık olarak tartışılır.

1.1 Amaç: iç kalite

Refactoring’in amacı, kodun iç kalite özelliklerini daha iyiye götürmektir. Yazılım kalitesi üzerine pek çok tanım ve gruplama vardır; hoca bunların en basitini tercih ettiğini söyledi: kaliteyi iç ve dış kalite özellikleri olarak ikiye ayırmak. Farklı standartlar farklı gruplamalar yapsa da alt kategoriler hepsinde aynıdır.

- **İç kalite özellikleri** yazılımı geliştirenleri ilgilendirir.

  - Taşınabilirlik: yazılımın mümkün olan en az değişiklikle başka platformlara taşınabilmesi. Bu işi geliştiriciler yapacağı için iç kalite özelliğidir; iş ne kadar kolaysa bu özellik o kadar iyidir.
  - İlk haftalarda konuşulan bağlaşım (coupling) ve uyum (cohesion).
  - Anlaşılabilirlik: okunan kaynak kodun mümkün olduğunca rahat anlaşılması.
- **Dış kalite özellikleri** yazılımı kullananları ilgilendirir.
  - Verimlilik: sistem kaynaklarını en az kullanan kod son kullanıcıyı ilgilendirir, çünkü yazılımın daha düşük kapasiteli sistemlerde de çalışabilmesini sağlar.
  - Kullanım kolaylığı.

Refactoring iç kalite özelliklerini düzeltmeye yöneliktir. Bunun bazı dış kalite özelliklerine de olumlu etkisi olabilir, ama hedef iç kalitedir.

Hoca anlaşılabilirliğe özellikle değindi: ileride görülecek refactoring eylemlerinden biri, bir üyenin adını daha anlamlı bir adla değiştirmektir. Bu kadar basit bir şey bile bir refactoring eylemidir ve yararı büyüktür, çünkü kodun o kısmının daha iyi anlaşılmasını sağlar. Refactoring ile koda esneklik kazandırılabilir, yeniden kullanılabilirlik artırılabilir, bağlaşım ve uyum iyileştirilebilir.

2. Refactoring Nasıl ve Ne Zaman Yapılır?

Slayttaki özet şöyledir (ekrandan, 00:10:00):

- Çevik yaklaşımla yapılır:
  - Mevcut gereksinimleri karşılayacak ilk tasarım yapılır.
  - İlk tasarımın mükemmel olması ve her bilinmezi öngörmeye çalışması gerekmez.
  - Yeterince iyi olan tasarım gerçeklenmeye başlanır.
  - Önceki tasarım adımlarından birinde yanlış karar verildiği anlaşılınca refactoring yapılır.
- Sürekli sınama ile yapılır:
  - Yapılan değişikliklerin yeni hatalar oluşturmadığından emin olmak için her değişikliğin ardından kodun yeni hâli sınanmalıdır.
- Ne zaman refactoring yapılır?
  - Kod derlenip toparlanmaya gerek duyduğu zaman: yeni özellikler eklemekte zorlanmaya başlandığında; bir hatayı bulmakta zorlanıp, bulunca da düzeltmek için pek çok ayrı yeri kurcalamak zorunda kalındığında.
  - Kod kötü kokmaya başladığında.

2.1 Çevik yaklaşım

Yazılım projelerinin doğası gereği gereksinimler değişir. Proje çok kısa sürmüyorsa, altı ay içinde bile yazılımın hitap ettiği problem alanında gereksinimler, kurallar, yönetmelikler ve ihtiyaçlar değişebilir. Bu yüzden ilk tasarımda olabilecek her şeyi düşünmeye çalışmak yerine, bugün kararlaştırılan gereksinim kümesini en iyi biçimde modellemek önerilir. İleride değişiklik çıkarsa refactoring eylemleriyle karşılanır.

Hoca burada “mükemmel, iyinin katilidir” sözünü hatırlattı: gereksinimleri karşılayan iyi bir tasarım yeterlidir.

2.2 Sürekli sınama

Refactoring küçük küçük değişikliklerle yapılan bir eylem olsa da küçük bir değişiklik bile sistemde öngörülemeyen yeni hatalar ortaya çıkarabilir. Bu yüzden her refactoring eyleminin ardından ilgili birim sınamalarını (unit test) yeniden çalıştırmak çok yararlıdır. Hoca bu nedenle derste birim sınaması için pratik bir aracın kullanımını da gösterdi (Bölüm 4).

2.3 Ne zaman?

Refactoring düzenli olarak da yapılabilir. Çevik süreçlerde çalışılıyorsa, bir sprintte ya da bir çevrimde yeni bir özellik eklemek (inkrement) yerine bir iyileştirme iterasyonu yapmak tercih edilebilir.

Bu kararın asıl işareti şudur: koda yeni özellik eklemekte ya da hataları düzeltmekte zorlanılmaya, bunlar için giderek daha uzun süre harcanmaya başlandığında kod derlenip toparlanmaya ihtiyaç duyuyordur. Başka bir deyişle, kod kötü kokmaya başlamıştır.

3. Code Smell (Kod Kusuru)

3.1 Terimin çevirisi

Hoca "code smell" terimini "kod kokusu" yerine **kod kusuru** diye çevirmeyi tercih ettiğini söyledi. İngilizcede "it is smelly" ya da "it is smelling something" genelde olumsuz anlamda söylenir. Ama kod kusuru bir hata değildir; bir bug ortaya çıkarmaz. Yalnız esnek olmayan ya da iç kalite ölçütleri düşük kod ortaya çıkaran durumlardır. Hoca benzetmesini şöyle kurdu: "Güzel bir insandır ama ufak bir kusuru vardır" denir; buna hata denmez, kusur denir.

> **Hocanın vurgusu:** Derste terimler Türkçeye çevrilse de bütün kod kusurları ve refactoring eylemleri literatürdeki özgün adlarıyla anılacaktır. Sınavlarda da bu özgün adlar beklenecektir.

3.2 Tanım

Slayttaki tanım şöyledir (ekrandan, 00:20:00):

- Yazılan kodu iyileştirmek gerektiğinin işaretleridir.
- Kod içerisinde karşılaşılan tersliklerdir.
- Kullanılan programlama yaklaşımının kusurlu kullanım örnekleridir.
- Zamanla bir yazılım yamalı bohçaya dönüşebilir:
  - Değişik zamanlarda farklı kişilerce kodun değişik kısımlarında değişiklikler yapılır.
  - Bu değişiklikler birlikte düşünülerek planlanabilseydi, daha iyi bir çözüme ulaşılması mümkün olabilirdi.
- Yazılım elden kayıp gitmeden tehlike işaretlerini sezip önlem almak yerinde olacaktır.

Programlama yaklaşımının kusurlu kullanımına en temel örnek şudur: önce yapısal programlama, ardından nesneye yönelik programlama öğrenilmişse, nesneye yönelik tasarımda ya da kodda, nesneye yönelik yaklaşımın önerdiği en iyi uygulamaların aksine yapısal programlama düşünce tarzıyla karar vermek.

İkinci kaynak zamandır. Ne kadar esnek kod yazmaya, tasarım kalıplarıyla yakın vadedeki değişikliklere karşı bağışıklık kazanmaya çalışılsa da, yapılan değişiklikler zamanla yazılımı bir yamalı bohçaya çevirebilir. Şu durumlar bunun işaretidir:

- Bir değişiklik yapmak için kendimizi kodun değişik noktalarında dolaşırken buluyorsak.
- Bir değişiklik için çok farklı yerlerde farklı işlemler yapmak gerekiyorsa.
- Denetim akışı incelendiğinde, akışın çoğu yerde gereksiz ve karmaşık kod parçalarından geçtiği, aslında daha sade yapılabileceği fark ediliyorsa.

Bu durumda kod yamalı bohçaya ya da spagetti koda dönüşmüş denebilir. Proje elden kayıp gitmeden bu tehlike işaretlerine önlem almak yararlıdır.

3.3 Code smell ve refactoring ilişkisi

Kodda karşılaşılan tersliklerin düzeltilmesi için yeniden yapılandırma eylemleri yürütülür. Derste her kod kusurunun adı, özellikleri, belirtileri ve nasıl refactor edileceği analitik bir yaklaşımla görülecektir. Bir kod kusurunu gidermek için yapılan refactoring eylemi başka bir kod kusuruna yol açabilir; bu fark edilirse yeni kusuru düzeltmek için yeni refactoring eylemleri yapılır (ekrandan, 00:20:00).

4. JUnit ile Birim Sınaması

4.1 Birim sınaması nedir?

Hoca yazılım sınamasının başlı başına bir uzmanlık alanı olduğunu, bu konuda ayrı dersler de bulunduğunu söyledi. Kısaca: en küçük yazılım bileşeninin sınanmasına birim sınaması (unit testing) denir. Herhangi bir programlama yaklaşımında birim sınaması yapılabilir; önemli olan,

o yaklaşımda en küçük yazılım biriminin ne olduğuna bakmaktır. Nesneye dayalı programlamada bu, bireysel sınıfların sorumluluklarının sınanmasıdır (ekrandan, 00:30:00).

Kim hazırlar? Birim testlerini programcı ekip, yani teknik ekip tasarlar. Ayrı bir sınama ekibi varsa o da katılabilir: kodun ana tasarımı, sınıfların üye alanları, üye metotları ve sorumlulukları belli olduktan sonra sınama ekibi bu tasarım üzerine birim testleri hazırlayıp kod havuzuna koyabilir. Her birimi kodlayan programcılar da sınama senaryolarını birim testleri biçiminde havuza ekleyebilir. Bu testler otomatik test araçlarıyla (JUnit bunlardan biridir) otomatik olarak çalıştırılıp değerlendirilir.

Ne zaman tasarlanır? Her aşamada: kodlamadan önce, kodlama sırasında ya da kodlamanın ardından. Kodlamadan önce yazmayı Test Driven Design (TDD) yaklaşımı önerir: bir satır bile üretim (production) kodu yazmadan önce tasarımdan yola çıkarak birim sınamaları yazılır ve yazılacak kod yalnız bu sınamaları geçmeye yönelik olur. Hoca bu yaklaşımın kendine göre avantajları ve dezavantajları olduğunu, belki herkese ve her projeye uygun olmadığını, ama bu tarzla çalışmaya başlayanların genelde onu çok benimsediğini söyledi. Testler kodla paralel de yazılabilir, kod bittikten sonra da. Hangisinin daha iyi olduğu tartışmasına girmedi: "Yeter ki yapılsın."

> **Ders sonrası not:** Hoca TDD'yi "Test Driven Design" diye açtı. Literatürde yaygın açılımı "Test-Driven Development"tır; büyük olasılıkla aynı yaklaşım kastedilmektedir.

4.2 Henüz kodlanmamış işbirlikçiler: vekil kod

Bir birimin sınaması yapılırken, o birimin bir sorumluluğunu yerine getirmek için işbirliği yaptığı, yani bağlaşımda olduğu diğer birimler henüz hiç kodlanmamış olabilir. Bu durumda sınamadan vazgeçilmez ya da ertelenmez. Bağlaşımda olan diğer sınıflar için, onların kodunun yerine geçecek **iskelet kod** yazılır. Örneğin işin başarılı olduğu durumu simgelemek için `return true` diyen bir iskelet yazılabilir. O birimin kodu tamamlandığında onun da kendi birim testleri çalıştırılacaktır. Daha sonra tümleştirme sınamaları aşamasında, ayrı ayrı sınanan birimlerin birlikte doğru çalışıp çalışmadığı da sınanır. Bu yüzden birim testlerini erkenden tasarlamaya ve kodlamaya çekinmeden başlanabilir.

Slayttaki ifade: bir sınıfın tek başına yürütemediği sorumlulukların sınanması için, bu sınıfın ihtiyaç duyduğu diğer sınıfların yerine geçecek kod gerekebilir. Bunlara vekil, sahte, yalancı kod/sınıf denir (stub, dummy, surrogate, proxy). Vekil yalnız ihtiyaç duyulan sınıflar gerçeklenene dek kullanılır (ekrandan, 00:30:00). Hoca bu adlar arasında bazı nüans farkları olduğunu, ama kendisinin bu konuda o kadar hâkim olmadığını, temelde "henüz kodlanmamış kodun yerine geçen kod parçası" diye özetlenebileceğini söyledi.

4.3 xUnit ailesi ve JUnit

Bu aşamada işi kolaylaştıran en yaygın test araçlarından biri xUnit ailesidir. X yerine kullanılan programlama ortamının harfi gelir: Java için JUnit, C# için csUnit gibi. JUnit, Java ile yazılmış kodun birim sınamaları için bir çerçeve programdır (framework) ve en yaygın kullanılan açık kaynak kod kitaplıklarından biridir.

IDE desteği (ekrandan, 00:30:00): Eclipse'te IDE ile hazır gelir (projede build path → add libraries ile JUnit'i eklemek için bir onay kutusu işaretlenir); NetBeans'te de hazır gelir. İstenirse en son sürüm junit.org sitesinden indirilip projeye dahil edilebilir.

JUnit'in en temel iki sürümü 3.x ve 4.x'tir. Hoca önce 3.x'i anlattı, sonra 4.x'in getirdiği yeniliklerden söz etti.

4.4 JUnit 3.x

Uygulama kodu `MyPackage` gibi bir pakette, JUnit kitaplığı `junit.framework` paketindedir. Uygulama mantığını içeren sınıflar `AClass1`, `AClass2` gibi sınıflar olsun.

Tercih edilen yöntem şudur:

- Her uygulama sınıfı için ayrı bir test sınıfı (bir test case ailesi) hazırlanır.
- Bu test sınıfları `junit.framework` paketindeki `TestCase` sınıfından kalıtımla türetilir.

- JUnit 3.x'te adlandırma zorunludur: bütün test case'leri içeren sınıfın adı `test` sözcüğüyle başlamalıdır. Örneğin `AClass1`'in birim sınamaları için `TestAClass1` sınıfı oluşturulur.
- Bu sınıfın içinde `testXxx` metotları yazılır. Her test metodu bir birim testine, bir test case'e karşılık gelir. Metodun adı da `test` ile başlamak zorundadır; geri kalanı testin ana amacını anlatan, gerekirse uzun bir addır.
- Test case sınıfları istenirse bir test takımında (test suite) toplanabilir.

Sonuç nasıl denetlenir? `TestCase` sınıfının ilişkide olduğu bir `Assert` sınıfı vardır ve metotlarının hepsi statiktir. Sınamanın en temel etkinliği kara kutu yaklaşımıdır: girdi verilir, beklenen doğru çıktı ile elde edilen sonuç karşılaştırılır. `assert` metotları bu değerlendirmeyi yapar:

- `assertEquals(beklenen, gerçekleşen)`: iki parametre de `Object` türündendir. Birinci parametre gerçekleşmesi beklenen doğru çıktı, ikincisi elde edilen sonuçtur. Eşitse test case başarılıdır.
- `assertNotEquals`: iki parametre eşit değilse başarılıdır.
- `assertTrue(ifade)`: örneğin bir metodun başarılı döndüğünü ya da bir bayrak değişkeninin doğru olduğunu denetlemek için. İfade `true` ise başarılıdır.
- `assertFalse(ifade)`: ifade `false` ise başarılıdır.
- `assertNull(nesne)`: nesne `null` ise başarılıdır; `assertNotNull` bunun tersidir.

Hoca bu varyasyonların hepsinin aslında `assertEquals` ile de yazılabileceğini söyledi. Örneğin `assertNull` için beklenen değer olarak `null` yazılır. Ama çerçeve işi kolaylaştırmak için ayrıntılı varyasyonları hazır sunar.

Sahneyi hazırlamak: `setUp()`. Test yapmadan önce teste katılacak nesneleri ilklendirmek, aralarındaki ilişkileri kurmak, gerekli set/get işlemlerini yapmak gerekir. Bunun için `TestCase` sınıfında korumalı (protected) bir `setUp()` metodu vardır; test sınıfında override edilir. Bir test sınıfında birden çok test metodu olur. Çerçeve, her test metodundan önce `setUp()`'ı otomatik olarak yeniden çalıştırır: önce sahne hazırlanır, birinci test çalışır; ikinci testten önce sahne yeniden hazırlanır, ve böyle sürer. Geliştiricinin yapması gereken yalnız `setUp`'ı override etmek, test metotlarını yazmak ve test sınıfını sağ tıklayıp "Run As → JUnit Test" ile çalıştırmaktır. `setUp`'ı elle çağırmaya gerek yoktur.

4.5 JUnit 4.x yenilikleri

Slayttaki özet (ekrandan, 00:40:00):

- Geriye doğru uyumluluk korunmakla birlikte, JUnit 4 sürümü ile annotation desteği gelmiştir (büyük/küçük harf duyarlılığı vardır).
- Artık sınama sınıflarının `TestCase` sınıfından kalıtımla türetilmesi gerekmemektedir, ancak şu eklemeler yapılmalıdır:

```
import static org.junit.Assert.*;
import org.junit.*;
```

- Artık sınama metotlarının adlarını `test` kelimesi ile başlatmak gerekli değildir; ilgili metotların başına `@Test` annotation koymak yeterlidir.
- `setup` adlı metodun yerine ise `@Before` annotation gelmiştir.

```
@Before
public void setUp() { /*Preparations*/ }
@Test
public void testSomething() { /*Do test*/ }
```

- Exception tanımlanan yazılımlarda atılması gereken exception'ların gerçekten ortaya çıkıp çıkmadığının sınanması da mümkün olmuştur:

```
@Test(expected=SomeException.class)
public void testTheException() throws SomeException {
    doSomethingThatCreatesTheException();
}
```

Hoca annotation'ların büyük/küçük harf duyarlı olduğunu, `@Test`'in ilk harfinin büyük, diğerlerinin küçük yazılması gerektiğini vurguladı. Adı ne olursa olsun başına `@Test` konan metot bir

birim sınaması vakası olarak ele alınır; başına `@Before` konan metot da setup metodu görevini yapar.

İstisna sınamasının önemi şudur: bazı durumlarda yazılımın tanımlı bir istisnayı fırlatması gerekir. `expected=` ile işaretlenen metotta belirtilen sınıftan bir istisna fırlatılırsa program akışı kesilmez; test case başarılı sayılır ve `setUp` tekrarlanarak sonraki testlerle devam edilir. İstisna fırlamazsa test başarısız sayılır, yürütme yine sonraki testle sürer. Ders örneklerinde istisna kullanılmayacağı için hoca kod örneklerini 3.x sürümüyle yaptı.

> **Ders sonrası not:** Hoca JUnit 3.x'te istisna testinin yapılamadığını, bir istisna olunca her şeyin yarıda kesildiğini söyledi (00:40:22). Büyük olasılıkla doğrusu: JUnit 3'te bunun için özel bir annotation yoktur, ama `try { doSomething(); fail(); } catch (SomeException e) { }` kalıbıyla istisna sınaması elle yazılabilir; beklenmeyen bir istisna da çerçeve tarafından yakalanıp yalnız o testin "error" sonucu olarak raporlanır.

5. Fowler'ın Video Kaset Kiralama Örneği

5.1 Kaynak ve anlatım düzeni

Kod kusurları ve refactoring konusu Martin Fowler'ın "Refactoring" kitabından alınmıştır; kitap ders notlarının başındaki kaynaklar arasındadır. Fowler kitapta önce bütüncül bir örnek üzerinde tek tek kod kusuru belirleyip refactoring eylemleri uygular, sonra kusurları ve eylemleri tek tek anlatır. Hoca bunun tersini tercih ediyor: önce kusurlar ve refactoring eylemleri, sonra bütüncül örnek üzerinde hepsinin uygulanması. Ancak her kod kusuru ve refactoring örneği de bu uygulamanın problem alanı üzerinden anlatıldığı için, örneklerin daha iyi anlaşılması amacıyla önce bu uygulama tanıtıldı.

5.2 Problem alanı

Ana tasarım basittir ama içinden epey malzeme çıkar. Bir video kaset kiralama işletmesinde müşteriler filmleri belirli süreliğine kiralar. Üç film türü vardır: normal filmler, yeni çıkan filmler ve çocuk filmleri. Farklı türdeki filmlerin farklı sürelerde kiralanması karşılığında farklı ücretler ödenir. Ayrıca kiralama işlemlerinden bonus puan kazanılır.

`Customer` sınıfının `statement()` metodu bütün bunları listeler ve bir makbuzun içine basılacak dizgiyi verir: "şu türden şu filmi şu kadar gün kiraladınız, borcunuz şu; … toplam borcunuz şu; kazandığınız toplam bonus şu".

İlişkiler: bir müşterinin n adet kira kaydı vardır; bir kira kaydı bir film ile ilişkilidir.

5.3 `statement()` metodunun akışı

Hoca Fowler'ın sıralama şemasını yeniden çizdi; kitaptaki versiyon yalnız metot çağrılarını gösteriyordu, geri dönüşleri ve nerede ne yapıldığını göstermiyordu. Akış şöyledir:

```
Customer.statement() çağrılır
  her kira kaydı için:
    Rental'dan film (Movie) alınır
    Movie'den fiyat kodu (priceCode) sorulur
    Rental'dan kaç günlük kiralandığı alınır
      -> bu kiralamanın ücreti hesaplanır
    Movie'den fiyat kodu yeniden istenir
    Rental'dan gün sayısı yeniden alınır
      -> bu kiralamanın bonusu hesaplanır
    Movie'den getTitle() çağrılır (makbuza "şu filmden şu borç" yazmak için)
  biriktirilen bilgiyle makbuz tamamlanır ve döndürülür
```

5.4 Kaynak kod

Kod Eclipse'te `refactoring/fowler_00` paketindedir; her yeni refactoring eyleminde paket numarası ilerletilecektir. `fowler_00` hiçbir refactoring yapılmamış ilk hâldir (ekrandan,

00:50:00).

`Customer` sınıfı (ekrandan, 00:50:00):

```
public class Customer {
    private String _name;
    private Vector<Rental> _rentals = new Vector<Rental>();
    public Customer (String name){
        _name = name;
    }
    public void addRental(Rental arg) {
        _rentals.addElement(arg);
    }
    public String getName (){
        return _name;
    }
    public String statement( ) {
        ...
    }
}
```

Kira kayıtları jenerik bir `Vector` üzerinde tutulur; `addRental` yeni kira kaydını vektöre ekler.

Alt çizgi gösterimi. Üye alanların adları alt çizgi ile başlar. Bunun nedeni şudur: IDE'de renklendirme sayesinde hangi sözcüğün üye alan, hangisinin parametre olduğu anlaşılır; ama kitapta ve slaytlarda kod kusurları çoğu kez bütün kod üzerinden değil yalnız ilgili parça üzerinden anlatılır ve "önceki hâl / sonraki hâl" karşılaştırması yapılır. Üye alanların kolay anlaşılması için C dilinde de kullanılan bu adlandırma notasyonu Fowler'ın Java örneklerine de yansımıştır. Alt çizgiyle başlayan bir ad görülürse bu bir üye alandır (ekrandan, 01:30:00).

`statement()` metodu. Upuzun bir metottur; birçok işin burada toplandığı anlaşılır. Toplam borç için kümülatif toplam tutan `totalAmount` ve bonus için `frequentRenterPoints` geçici değişkenleri tanımlanır. (Fowler "frequent flyer" benzetmesiyle bu adı seçmiştir.) Kira kayıtları bir `Enumeration` ile `while` döngüsünde dolaşılır. Önce makbuzun ilk satırı oluşturulur: `"Rental Record for " + getName() + "\n"`. Her kira kaydı için o kaydın borcunu tutacak `thisAmount` tanımlanır ve fiyat kodu üzerinden bir `switch` kurulur (ekrandan, 00:53:10):

```
while (rentals.hasMoreElements()) {
    double thisAmount = 0;
    Rental each = rentals.nextElement();
    //determine amounts for each line
    switch (each.getMovie().getPriceCode()) {
    case Movie.REGULAR:
        thisAmount += 2.0;
        if (each.getDaysRented() > 2)
            thisAmount += (each.getDaysRented() - 2) * 1.5;
        break;
    case Movie.NEW_RELEASE:
        thisAmount += each.getDaysRented() * 3.0;
        break;
    case Movie.CHILDRENS:
        thisAmount += 1.5;
        if (each.getDaysRented() > 3)
            thisAmount += (each.getDaysRented() - 3) * 1.5;
        break;
    }
    // add frequent renter points
    frequentRenterPoints ++;
    // add bonus for a two day new release rental
    ...
}
```

Ücret kuralları:

- Normal film: kiralamada 2 lira borç yazılır; bu 2 gün kiralama hakkı verir. 2 günü aşan her gün için gün başına 1,5 lira eklenir.
- Yeni çıkan film: her gün için 3 lira.
- Çocuk filmi: 1,5 lira; bu 3 gün saklama hakkı verir. 3 günü aşan her gün için gün başına 1,5 lira eklenir.

Puan kuralları: her kiralama için 1 puan. Ayrıca yeni çıkan bir film 2 gün ya da daha uzun kiralanmışsa 1 puan daha. Döngünün sonunda filmin adı ve borcu makbuza eklenir, kümülatif toplam güncellenir. Bütün kira kayıtlarından sonra toplam borç ve kazanılan bonus yazılıp sonuç döndürülür.

Hoca kodda `System.currentTimeMillis()` ile başlayan bir zaman ölçümü satırı eklediğini fark etti (ekrandan, 00:50:00). Bunu refactoring ile optimizasyon arasında bazen bir çelişki olabildiğini göstermek için koyduğunu, özgün kodda ve kitapta olmadığını söyleyerek şimdilik kapattı; ileride yeniden ekleneceğini belirtti.

`Rental` ve `Movie`. Diğer sınıflar hafiftir. `Rental` kira kaydının hangi filmle ilişkili olduğunu ve kaç gün kiralandığını tutar; kurucu ikisini de alır, ikisinin getter'ı vardır. `Movie`'de film türleri `public final static int` üye alanlarla kodlanmıştır: `REGULAR = 0`, `NEW_RELEASE = 1`, `CHILDRENS = 2`. Hoca bunun tam bir `enum` olmadığını, ama enum gibi kullanıldığını söyledi. Filmin adı ve fiyat kodu üye alanlardır; kurucu ikisini alır; ikisinin de getter'ı vardır. Fiyat kodunun bir setter'ı da vardır, çünkü bir filmin kodu değişebilir: yeni çıkan bir film belli bir süre sonra (altı ay, bir yıl, her neyse) normal filme çevrilir; ya da bir film yanlışlıkla çocuk filmi olarak işaretlenmiş olabilir.

Hoca önümüzdeki derslerde refactoring eylemlerinin bütün örnek üzerinde uygulanacağını, şimdi tanıtılan uygulamanın bu yüzden önemli olduğunu söyledi.

6. Örnek Testler ve Çalıştırılması

6.1 `TestCustomer`

En çok `Customer`'ın, özellikle `statement()` metodunun test edilmesi gerekir. Bunun için farklı türdeki filmler farklı sürelerde kiralanır ve doğru makbuzun, yani doğru ücretin ve doğru puanın üretilip üretilmediğine bakılır. Bu elle yapılmaz: her bakılacak şey için ayrı bir test metodu hazırlanır. Hepsinden önce bir müşteri ve çeşitli türlerden filmler oluşturmak gerektiği için `setUp()` override edilir. `TestCustomer`, `junit.framework.TestCase`'ten kalıtımla türetilmiştir (ekrandan, 01:00:00):

```
protected void setUp() {
    yunus = new Customer("Yunus Emre Selçuk");
    matrix = new Movie("The Matrix",Movie.REGULAR);
    monster = new Movie("Monsters, Inc.",Movie.CHILDRENS);
    surrogate = new Movie("Surrogates", Movie.NEW_RELEASE);
    terminator = new Movie("Terminator Salvation",Movie.NEW_RELEASE);
}

public void testGetName( ) {
    String sonuc = yunus.getName( );
    assertEquals("Yunus Emre Selçuk",sonuc);
}

public void testStatementWhenEmpty( ) {
    String sonuc = yunus.statement();
    String beklenen = "Rental Record for Yunus Emre Selçuk\n";
    beklenen += "Amount owed is 0.0\n";
    beklenen += "You earned 0 frequent renter points";
    assertEquals(beklenen, sonuc);
}
```

`testGetName` basit görünebilir. Ama hoca bunun "dudak kıvrılacak" bir test olmadığını söyledi: müşteri adında Türkçe karakter geçtiği için, sistemin Türkçe karakterleri doğru saklayıp doğru geri verip vermediği de bir taşla iki kuş vurularak sınanmış olur.

`testStatementWhenEmpty`: müşteri oluşturulmuş ama hiç kiralama yapmamıştır. Müşteriye kafadan bir borç yazılmamış, kafadan bedava bir puan da verilmemiş olmalıdır; borç ve puan sıfır olmalıdır. Beklenen makbuz dizgisi oluşturulur ve `assertEquals` ile karşılaştırılır.

Uzun kiralama testi (ekrandan, 01:01:30):

```
public void testStatementWithMoviesLongRent( ) {
    yunus.addRental( new Rental(matrix, 3) );
    yunus.addRental( new Rental(monster, 4) );
    yunus.addRental( new Rental(surrogate, 2) );
    String sonuc = yunus.statement();
    String beklenen = "Rental Record for Yunus Emre Selçuk\n";
    beklenen += "\tThe Matrix\t3.5\n";
    beklenen += "\tMonsters, Inc.\t3.0\n";
    beklenen += "\tSurrogates\t6.0\n";
    beklenen += "Amount owed is 12.5\n";
    beklenen += "You earned 4 frequent renter points";
    assertEquals(beklenen, sonuc);
}
```

Hesap adım adım (ücret kuralları Bölüm 5.4'teki koddan):

- The Matrix, normal film, 3 gün: $2 + (3 - 2) \times 1{,}5 = 3{,}5$.
- Monsters, Inc., çocuk filmi, 4 gün: $1{,}5 + (4 - 3) \times 1{,}5 = 3{,}0$ (ara adım eklendi).
- Surrogates, yeni film, 2 gün: $2 \times 3 = 6{,}0$.
- Toplam: $3{,}5 + 3{,}0 + 6{,}0 = 12{,}5$.
- Puan: üç kiralama için 3 puan, yeni filmin 2 günlük kiralanmasından 1 bonus puan; toplam 4.

Hoca test metodunun adının uzun olduğuna dikkat çekti: ad, metotta ne yapılmaya çalışıldığını açıkça anlatır ve kodun anlaşılabilirliğini artırır.

> **Hocanın vurgusu:** Uygulama mantığı doğru kodlanmış, ama test case yanlış kodlanmış olabilir; hata ondan kaynaklanıyor olabilir. Bu yüzden testin ne yaptığını anlatan, varsın uzun olsun, ayrıntılı bir ad vermek iyi bir uygulamadır.

Diğer testler: `testStatementWithMoviesShortRent` farklı türdeki filmleri bu kez kısa süreli kiralar. `testNewReleaseRentalBonus` yeni filmlerde ek puan kazanılıp kazanılmadığını sınar: bir yeni film 2 günlüğüne (2 puan), bir normal film 1 günlüğüne (1 puan) kiralanır; toplam 3 puan beklenir.

6.2 Çalıştırma: yeşil ve kırmızı çubuk

"Run As → JUnit Test" denince JUnit testleri işler. Yeşil çubuk, bütün testlerin çalıştığını ve her şeyin yolunda olduğunu gösterir; altta hangi test case'lerin çalıştığı listelenir.

Hoca bir hatanın nasıl görüneceğini canlandırdı: normal filmin kirasında ilk başta 2 lira, sonra her gün için 1,5 lira olması gerekirken, gereksinimi yanlış anlamış gibi günlük ek ücreti 2,5 yaptı. Testler yeniden çalıştırılınca kırmızı çubuk göründü. Sorun `testStatementWithMoviesLongRent` test case'inde işaretlendi. JUnit, normalde eşit olması gereken dizgilerin nerede farklılaştığını köşeli parantezle gösterdi: beklenen "The Matrix 3.5", elde edilen "The Matrix 4.5". Gerçekten $2 + (3 - 2) \times 2{,}5 = 4{,}5$ olur. Matrix normal bir film olduğu için hata normal film ücretinin hesabındadır; hoca gereksinim belgesine bakıp kodu düzeltti, testler yeniden yeşil çubuk verdi. Çerçeve, hatayı bulmak için yapılacak işi geliştiricinin yerine yapmış olur.

6.3 `TestMovie` ve açıklama dizgisi

`TestMovie`'de üç farklı türden film (regular, children, new release) oluşturulur. Farklı türdeki filmlerin fiyat kodları karşılaştırıldığında eşitlik çıkmaması beklenir; `assertNotEquals` kullanılır. Bu sınıfta tek test metodu olduğu için `setUp` hazırlanmadı, ama tek bir test metodunda birden fazla `assert` kullanıldı. Hoca, önerilen uygulamanın bir test metodunda tek bir `assert` kullanmak olduğunu, bunu yalnız farklı yapılabileceğini göstermek için böyle yaptığını söyledi. Metottaki üç `assert`'ün hepsi doğru olduğu için çubuk yeşildir; biri bile tutmasaydı kırmızı olurdu.

Bu testte `assert` metotlarına ilk parametre olarak bir dizgi verildi. Bütün `assertXxx` metotlarının, ilk parametre olarak bir açıklama dizgisi alan aşırı yüklenmiş (overloaded) sürümleri vardır. Test başarısız olursa, JUnit yalnız farklılaşan noktayı göstermekle kalmaz, bu açıklama metnini de hata iletisinin başına ekler.

6.4 Test takımı: `TestAll`

`TestMovie` ile `TestCustomer`'ı bir takım (suite) olarak birlikte çalıştırmak için `TestAll` adlı bir sınıf hazırlandı. Bu sınıfta belirli bir imzaya sahip bir `suite()` metodu yazılır; `TestSuite` kullanılarak her test sınıfının `.class` değeri eklenir. Sağ tıklanıp çalıştırıldığında toplam 6 test case çalışır (ekrandan, 01:10:00): `TestCustomer` altında `testGetName`, `testStatementWhenEmpty`, `testNewReleaseRentalBonus`, `testStatementWithMoviesShortRent`, `testStatementWithMoviesLongRent`; `TestMovie` altında `testMovie`. Hepsi geçtiği için yeşil çubuk görülür.

6.5 Soru-cevap: yerel kodlu bir bileşenin sınanması

Soru: Ayrı bir donanım (örneğin gömülü bir cihaz) ya da telefondaki yüz tanıma gibi yerel (native) bir bileşen yalnız `true` ya da `false` döndürüyorsa, bunu nasıl test ederiz? PIN'in doğru olup olmadığı gibi işlemler kolayca test edilebiliyor, ama yüz tanındığında `true` dönecek bir tetikleme yapılamıyor. Yalnız test aşamasında çalışan bir alan (field) koymak doğru olur mu?

Cevap: Olabilir. Ya da kaynak koduna sahip olunmayan uygulamanın dışarıya bir biçimde sinyal vermesi gerekir; belki bir metin dosyasına yazması gibi. Hoca bunun biraz farklı bir uygulama olduğunu, kesin bir yanıtı olmadığını ve bir test uzmanına sorulması gerektiğini söyledi. Bunun için tümleştirme testlerine yönelik başka araçlar vardır. Yüz tanıma uygulamaları genelde önce görüntüde bir yüz olup olmadığını yakalar, onu bir çerçeveye alır, sonra bir etiket (kişinin adı) koyar. Canlı görüntü üzerinde etiketin doğru olup olmadığını anlık test etmeyi otomatikleştiren bir aracı hoca bilmediğini söyledi.

7. Duplicated Code ve Extract Method

7.1 Ele alınan kusurlar ve eylemler

Bundan sonraki yaklaşım şudur: önce bir kod kusuru tanıtılır, sonra o kusuru düzeltmeye yönelik bir ya da daha fazla refactoring eyleminden söz edilir. Hoca Fowler'ın kitabındaki bütün kod kusurlarını ve refactoring eylemlerini almadığını, dersin akışına göre bir seçim yaptığını söyledi. Tasarım kalıplarından sonra dönemin kalan süresinde bütüncül olarak anlatılabilecek kadarını düşünerek kod kusurlarının yaklaşık yarısını (sekiz tanesini), refactoring eylemlerinin biraz daha fazlasını seçmiştir. Daha fazlası için kitaba başvurulabilir.

Kod kusurları ile refactoring eylemleri arasında birebir değil, çoka-çok (N–M) bir eşleşme vardır. Hoca seçimi şöyle yaptığını anlattı: Fowler'ın kitabındaki sıradan başlayıp ipin ucunu izleyerek, bir refactoring eyleminden söz ettikten sonra o eylemin çözdüğü başka kod kusurları varsa onları da ders notuna ekleyerek bir öbek oluşturdu.

Fowler'ın kitabındaki düzen derste de izlenir: refactoring eylemi tanıtıldıktan sonra eylem yapılmadan önceki ve yapıldıktan sonraki durum yan yana gösterilir ("önce / sonra" karşılaştırması; hoca bunu reklamlardaki önce-sonra görüntülerine benzetti).

7.2 Code smell: Duplicated Code (yinelenen kod)

Slayttaki tanım (ekrandan, 01:30:00):

- Aynı kodun veya çok benzer kod yapısının birden fazla yerde tekrarlanması durumudur.
- Bu kodun işlevselliğinde değişiklik yapılması gerektiğinde, kodun tekrarlandığı her yerde aynı işlemlerin tekrarlanması gerekecektir.
- Ortaya çıkan sonuç bir bakım kabusudur.
- Çözüme götürecek düzenlemeler (refactorings):
  - Extract Method: yinelenen kodu bir metot altında toplamak amacı ile kullanılır.
  - Yinelenen kodu toplamaya yönelik diğer düzenlemelere ileride değinilecektir.

Hoca yinelenen kodun büyük olasılıkla kopyala-yapıştırdan doğduğunu söyledi: bir yerden alınıp öbür tarafa yapıştırılan kod, bazen hiç değiştirilmeden, bazen ufak düzenlemelerle kullanılır.

> **Hocanın vurgusu:** Kod kusurlarının özgün adlarını hatırlamanız önemlidir; sınavlarda bunlar üzerinden konuşulacaktır.

7.3 Refactoring: Extract Method (metot çıkartma)

Slayttaki tanım (ekrandan, 01:40:00):

- Bir metot içindeki kodun bir kısmını yeni bir metot olarak belirlemeye dayanır.
- Bu yapılandırma birçok kod kusurunu (smell) düzeltir.
- Oluşturulan yeni metoda, amacını açıkça belirten isabetli bir ad verilmelidir. Böylece alt düzey ayrıntılarla uğraşmadan kodun anlaşılabilirliği artar.

Extract Method yalnız Duplicated Code'u değil, birçok kod kusurunu düzeltir; ilgili kusurlar görüldükçe adı yeniden anılacaktır. Yeni metodun adı, gövdesine girmeden ne yaptığını anlatabiliyorsa kodun anlaşılabilirliği artar ve bu bakım masraflarını düşürür.

Örnek (ekrandan, 01:40:00):

```
/*** ÖNCE: ***/
void printOwing(double amount) {
    printBanner();
    //print details
    System.out.println("name:" + _name);
    System.out.println("amount" + amount);
}

/*** SONRA: ***/
void printOwing(double amount) {
    printBanner();
    printDetails(amount);
}
void printDetails(double amount) {
    System.out.println("name:" + _name);
    System.out.println("amount" + amount);
}
```

Önceki hâlde, önce makbuzun başlığı basılır, sonra bir açıklama yorumu konmuş, ardından `_name` üye alanı ve `amount` miktarı yazdırılmıştır.

Yorum mu, iyi ad mı? Hoca, kodu mümkün olduğunca yorumlamanın öğretildiğini, ama burada bunun bir "gömlek üstünü" söylediğini belirtti: metotları, değişkenleri, üye alanları ve parametreleri öyle adlandırın ki yorum yazmak zorunda bile kalmayın; kod kendini anlatsın. Bunun için böl-parçala-yönet mantığıyla alt sorumlulukları kendi metotlarına çıkarmak ve bu metotlara anlamlı adlar vermek gerekir.

Ne yapıldı? Ayrı bir açıklama yorumunun bulunması, oradaki işlemlerin yukarıdakilerle ilişkili olmakla birlikte kendi içinde bir bütün oluşturduğunu, yani metodun simgelediği işin alt adımlarından birini oluşturan bir öbek olduğunu gösterir. Bu öbek ayrı bir metoda kodlandı ve adı `printDetails` oldu. `printOwing(double amount)` imzası aynı kaldı, çünkü refactoring bunu ister: dışarıya sunulan arayüz değişmeden iç kod değişir. `printBanner()` çağrılmaya devam eder, ek olarak yeni çıkarılan `printDetails` çağrılır.

Bir ayrıntı: çıkarılan kısım metoda gelen `amount` parametresine bağımlıdır. Bu yüzden yeni metoda da `amount` parametresi verilmeli ve `printOwing` bu parametreyi ona iletmelidir. Böylece hiçbir hataya yol açmadan kod sağdaki hâle getirilir.

Kazanımlar. Daha modüler, daha odaklanmış bir kod elde edildi; metotların kendi içindeki uyum arttı. Nesneye yönelik yaklaşımın önerdiği böl-parçala-yönet mantığına da uyuldu. Gerçek bir uygulamada ayrıntıları yazmak bu iki satırdan çok daha uzun ve karmaşık olur. Bir hata ya da gereksinim değişikliği olduğunda:

- Eski hâlde, uzun metodun içinde değişikliğin nerede yapılacağını satır satır inceleyerek aramak gerekir.
- Yeni hâlde önce ana metoda (`printOwing`) gidilir; orada `printBanner` ve `printDetails` çağrıları görülür. Düzeltme ayrıntılarla ilgiliyse doğrudan `printDetails`'e tıklanıp daha odaklanmış bir yerde aranır; banner ile ilgiliyse `printBanner`'a gidilir.

Böylece ilk adımda arama uzayı daraltılmış, angarya iş azaltılmış ve asıl yapılması gereken işe daha çabuk ulaşılmış olur. Refactoring'in kazanımı budur.

Hoca bu noktada, konunun başında olunduğu için eylemlerin henüz çok tartışmalı (provokatif) olmadığını, ilerledikçe daha tartışmalı hâle geleceğini söyleyerek soru ve itiraz istedi.

8. Long Method

8.1 Code smell: Long Method (aşırı uzun metot)

Hoca bu kusuru "aşırı uzun kod" diye çevirmek istediğini söyledi. Metodun uzunluğu arttıkça:

- Metodu anlamakta daha fazla güçlük çekilir.
- Metodun gereksinimlerden kaynaklanan değişikliklerden etkilenme olasılığı artar; bu artan olasılık nedeniyle düzeltme yapılması gerektiğinde, nerede düzeltileceğini anlamak da zorlaşır.
- Metodun içindeki odaklanmışlık, yani uyum düşer. Metot uzadıkça daha fazla iş yapılıyor demektir; işler çeşitlendikçe bu işlerin ilgi alanlarının birbirinden uzaklaşma olasılığı artar ve metodun uyumu azalma eğilimine girer.

Çözüm yine Extract Method'dur: uzun kod incelenir, ortak amaca yönelik ya da kendi içinde uyumlu kod öbekleri belirlenir ve kaç öbek bulunursa o kadar metot çıkarılır. Long Method'u düzeltmek için başka refactoring eylemleri de vardır (Bölüm 10).

8.2 Örnek: `printOwing()`

Önceki hâl (ekrandan, 01:50:00):

```
void printOwing( ) {
    Enumeration e = _orders.elements();
    double outstanding = 0.0;
    // print banner
    System.out.println("*****************");
    System.out.println("* Customer Owes *");
    System.out.println("*****************");
    // calculate outstanding
    while (e.hasMoreElements()) {
        Order each = (Order) e.nextElement();
        outstanding += each.getAmount();
    }
    //print details
    System.out.println ("name:" + _name);
    System.out.println ("amount" + outstanding);
}
```

Müşterinin borcu ekrana yazdırılacaktır. Siparişler bir `Enumeration` ile alınır; toplam borç için kümülatif toplam sıfırla başlatılır. Yorumlardan sonra banner basılır, toplam borç hesaplanır, sonra ayrıntılar yazılır. Aradaki boş satırlar hangi yorumun hangi kod parçasıyla ilişkili olduğunu gösterir. Yorumlar sayılmazsa 10 satırlık bir metottur; çok uzun değildir, ama gerçek uygulamada daha uzun olacaktır.

Yorumlardan üç ayrı grup olduğu görülür: banner basma, borç hesabı ve ayrıntı yazdırma. Ayrıntıların yazılabilmesi için toplam borcun belirlenmesi gerekir.

Adım 1: banner. İlk grup `printBanner()` metoduna çıkarılır. Parametre gerekmez, çünkü yalnız sabit bir formatı yazan düz bir metottur.

Adım 2: bağımlılıklar. Kalan kısım tek metoda toplanırsa yeterince ince taneli bir çıkarma yapılmış olmaz; burada birbirine bağımlı olsa da iki ayrı alt işlem vardır. Bağımlılıklara bakılır: borç hesaplanır, sonra yazdırma kısmında bu borca ihtiyaç duyulur. `_name` bir üye alan olduğu için her yerden erişilebilir; sorun yoktur. Ama `outstanding` bu metot içinde tanımlanmış bir geçici değişkendir ve yukarıda değeri değiştirilir. Ayrıntıları basan kısmın yalnız hesaplanmış sonucu bilmesi yeter. Bu yüzden önce hesaplama metodu oluşturulur: `getOutstanding()` sonucu döndürür. Geçici değişkene artık yalnız bu metodun içinde ihtiyaç vardır; ayrıntı basan kısma ise parametre olarak verilir.

Adım 3: döngü. `getOutstanding()` içinde `while` yerine for-each döngüsü kullanılır. Hoca bunun iki nedeni olduğunu söyledi: for-each kendisine daha kolay anlaşılır geliyor; ayrıca önceki kodda bir tür dönüşümü (typecasting) vardı. `Enumeration` jenerik olarak (`Enumeration<Order>`) yazılmadığı için `getAmount()` çağrısından önce `(Order)` dönüşümü gerekiyordu. For-each döngüsünde döngü değişkeni doğrudan `Order` türünden tanımlandığı için `getAmount()` çağrılabilir ve dönüşüm ortadan kalkar.

> **Hocanın vurgusu:** Bir yerde tür dönüşümü görüyorsanız, bu büyük ihtimalle aslında daha iyi bir tasarım yapılabileceği ya da kodun daha iyi yazılabileceği anlamına gelir; büyük olasılıkla çok biçimlilik kurularak. Burada buna bile gerek kalmadan, JDK 1.5 ve üstüyle gelen for-each döngüsü sade bir çözüm sağladı.

Hoca KISS ilkesini de hatırlattı: “Keep it simple and stupid”, yani işi basit tutmak gerekir, yoksa aptallık edilmiş olur.

Adım 4: ayrıntılar. `printDetails(double outstanding)` yazılır. Metodun adı ne işe yaradığını söylediği için yoruma gerek kalmaz; aynı şey `getOutstanding` için de geçerlidir.

Sonraki hâl (ekrandan, 01:50:00):

```
void printOwing( ) {
    printBanner();
    printDetails(getOutstanding());
}
void printBanner() {
    System.out.println("*****************");
    System.out.println("* Customer Owes *");
    System.out.println("*****************");
}
double getOutstanding() {
    double outstanding = 0.0;
    for (Order each : _orders)
        outstanding += each.getAmount();
    /* for-each ile döngü kısaldı! */
    return outstanding;
}
void printDetails (double outstanding) {
    System.out.println ("name:" + _name);
    System.out.println ("amount" + outstanding);
}
```

İç içe parantezli ifadelerde önce en içteki ifade değerlendirilir. Bu yüzden `printBanner()`’dan sonra önce `getOutstanding()` çağrılır, hesabı yapar ve sonucu `printDetails`’e verir; `printDetails` aldığı borç bilgisine göre ayrıntıları yazdırır.

8.3 Parça sayısının artması bir dezavantaj mı?

Önceki hâlde 10 satırlık uzunca bir metot vardı; şimdi daha basit, daha anlaşılır parçalar vardır. Dezavantaj olarak uğraşılacak parça sayısının arttığı düşünülebilir. Ama bütün parçalara birden ulaşmaya gerek yoktur: bir düzeltme yapılacaksa önce `printOwing`’e girilir, orada upuzun bir metot yerine üç metot çağrısı görülür. Bu üç metodun amaçları adlarından bellidir; düzeltme hangisiyle ilgiliyse doğrudan ona atlanır, diğer parçalarla uğraşmak gerekmez. Hoca bu yüzden parça sayısının artmasının aslında bir dezavantaj olmadığını söyledi.

Hoca bu dersi uzun süredir verdiği için itirazların ya da oturmayan yerlerin nerelerden geldiğini aşağı yukarı tahmin edebildiğini, bunları ders notlarına ve sözlü anlatıma eklediğini; ama yeni insanların yeni bakış açıları getirebileceğini söyleyerek soru sormaya çağırdı.

8.4 Küçük nesne ve metotların yararı ve başarıma etkisi

Büyük sınıflar ve az sayıda nesne yerine küçük sınıflar ve nesneler kullanılması tercih edilir. Bunun yararları:

- Uygulama mantığının, kendi içinde yüksek uyumlu, odaklanmış kısımlarının programın çeşitli yerlerinde paylaşılması kolaylaşır.
- Sınıfların anlaşılması kolaylaşır.
- Paylaşılan kısımlar belki başka uygulamalarda da yeniden kullanılabilir.
- Uyum doğal olarak yükselir.

Yönetilecek parça sayısının artmasının bir dezavantajı olarak başarım da sayılabilir. Bir metot içinden başka bir metot çağrıldığında, çağıran metodun durum bilgisi bir yığına atılır, yeni çağrılan fonksiyon için yeni bir durum bilgisi oluşturulur; o fonksiyon bitince yığının en üstündeki bilgi alınıp (pop) çağıranın bağlamına dönülür. Hoca buna “context switching” adını verdi. Bu bir ek yüktür. Ama:

- Bu iş onlarca kez yapılmıyor; örnekte bir metot üç ya da en fazla dört metoda bölündü, yani yalnız birkaç ek çağrı var.
- Modern derleyici ve yorumlayıcıların çoğu bu ek yükü büyük oranda optimize ediyor, sıfıra çok yakın hâle getiriyor.

Bu yüzden küçük nesne ve metotların kullanımı başarımı olumsuz etkileyen bir durum ortaya çıkarmaz.

> **Ders sonrası not:** Hoca metot çağrısında çağıran metodun durum bilgisinin “heap bellekte bir yığına” atıldığını söyledi (01:54:29). Doğrusu, bu bilgi (dönüş adresi, yerel değişkenler) çağrı yığınına (stack) konur; heap dinamik olarak oluşturulan nesnelerin bellek bölgesidir. Ayrıca “context switch” terimi işletim sistemlerinde genelde süreç ya da iş parçacığı değişimi için kullanılır; burada kastedilen büyük olasılıkla fonksiyon çağrısı ek yüküdür.
>
> **Ek bilgi (derste anlatılmadı):** Derleyicilerin bu ek yükü azaltmak için kullandığı başlıca tekniklerden biri satır içi açmadır (inlining): küçük bir metodun gövdesi çağrıldığı yere kopyalanır ve çağrı hiç yapılmaz. Java’da JIT derleyicisi sık çağrılan küçük metotları çalışma anında bu biçimde açabilir.

9. Yinelenen Kod İçin Diğer Çözümler: Pull Up Method ve Form Template Method

9.1 Seçenekler

Slayttaki özet (ekrandan, 02:00:00):

- Pull Up Method: yinelenen kod kalıtım ağacındaki kardeş veya kuzen sınıflara yayılmışsa, oluşturulan metot ortak üst sınıfa çekilir.
- Form Template Method: kod aynı değil ancak benzerse, GoF Template Method tasarım kalıbı kullanılarak kardeş/kuzen sınıflarda düzenleme yapılır.
- Eğer yinelenen veya benzer kod ilişkisiz sınıflarda yer alıyorsa:
  - Sorumluluklar doğru dağıtılmamış olabilir.
  - Metot bir sınıfa atanır, diğerleri bu sınıfı kullanır.
  - Ya da bu metot oluşturulacak yeni bir sınıfa atanır ve diğerleri bu sınıfı kullanır.
- Özetle, Extract Method ile oluşturulan yeni metoda en uygun yer aranır.

Refactoring adlarında “up” ya da “down” görülürse, bu kalıtım hiyerarşisi içinde yukarı ya da aşağı hareket etmek demektir (pull up, pull down gibi).

9.2 İlişkisiz sınıflarda yinelenen kod

Gerçek hayatta tekrar eden kod, aralarında bağlaşım olmayan sınıflarda da yer alabilir. Bu, sorumlulukların sınıflara doğru dağıtılmadığının bir işareti olabilir: o sorumluluğu yerine getirebilecek daha uygun bir sınıf vardır, ama görülememiş ve sorumluluk başka yerlere atanmıştır. Bu durumda tasarım gözden geçirilmelidir.

Ancak bu değişiklik refactoring'in ana ilkesi olan "dış arayüzü değiştirmeme" ilkesini bozabilir. Hoca bunun için Proxy tasarım kalıbını önerdi, ama kalıbı özellikle kurmak yerine onun yaptığı asıl iş olan **metot çağrısını iletme** olarak algılanmasını istedi:

- İlişkisiz sınıflardaki kod toplanıp tek bir sınıfa taşınır.
- Kodun alındığı diğer sınıflarda metodun adı ve imzası kalır; ama yaptığı tek şey, taşındığı yerdeki metodu çağırıp sonucu geri iletmektir. Böylece dış arayüz bozulmaz.
- Geriye doğru uyumluluğun korunması gerekmiyorsa, metot diğer sınıflardan tamamen kaldırılıp taşındığı yer üzerinden çağrılabilir.
- Ya da kod hiçbirine taşınmaz, yepyeni bir sınıf oluşturulur; diğer sınıflara bu yeni sınıftan bir nesneye işaret verilir ve bu yetki devri (delegation) onun üzerinden yürütülür.

Özetle Pull Up Method, Form Template Method ve Extract Method özünde aynı şeyi yapar: bir kod bloğunu çıkarır ve onu yerleştirecek en iyi yeri arar. En iyi yer kalıtım hiyerarşisinde bir üst sınıfsa iş Pull Up Method adını alır; aynı sınıfın içindeyse Extract Method; Template Method kalıbı uygulandıysa Form Template Method.

9.3 Refactoring: Pull Up Method (metodu yukarı çekme)

Kalıtım ilişkisi içindeki kardeş sınıflarda yinelenen metotlar bulunuyorsa, bunları üst sınıfa çekip tek bir yerde kodlamak daha avantajlıdır; metot kalıtımla alt sınıflara zaten aktarılır.

Örnek: `Employee` sınıfından kalıtımla `Salesman` ve `Engineer` türetilmiştir; her satışçı bir çalışandır, her mühendis de bir çalışandır. İkisinde de aynı `getName` metodu vardır; ikisi de adı aynı biçimde döndürür. Aynı işi yapan aynı gövde görüldüğüne göre metot ortak ata olan `Employee`'ye taşınır: çıkarılıp yukarı çekilir. Kazanım: bir `Employee` listesi içinde her çalışan türünün adına ulaşmak için ayrı ayrı ilgili türe dönüşüm (cast) yapmak gerekmez; hepsinin `getName` metodu çağrılır. Bazı çalışan türlerinde metodun farklı davranması gerekirse yalnız o türde override edilir.

Özel durumlar (ekrandan, 02:02:50). `Customer` sınıfı `lastBillDate` üye alanını ve yeni fatura eklemek için `addBill(dat: Date, amount: double)` metodunu tutar. `Regular Customer` ve `Preferred Customer` alt sınıflarında `createBill(Date)` ve belirli bir başlangıç ve bitiş tarihi arasında faturayı hesaplayan `chargeFor(start: Date, end: Date)` metotları vardır. İkisinde de aynı metot vardır, ama gövdeleri "çok farklı kodlanmış"tır.

```
ÖNCE:
  Customer { lastBillDate; addBill(dat: Date, amount: double) }
    ├── Regular Customer   { createBill(Date); chargeFor(start: Date, end: Date) }
    └── Preferred Customer { createBill(Date); chargeFor(start: Date, end: Date) }
    (iki chargeFor "çok farklı kodlanmışlar")

SONRA:
  Customer { lastBillDate; addBill(dat: Date, amount: double);
             createBill(Date); chargeFor(start: Date, end: Date) {soyut} }
    ├── Regular Customer   { chargeFor(start: Date, end: Date) }
    └── Preferred Customer { chargeFor(start: Date, end: Date) }
    (not: "chargeFor metodu üst sınıfta soyut tanımlandı")
```

Gövdeler çok farklıysa metot somut olarak yukarı taşınmaz; alt sınıflarda override edilip gövdeler olması gerektiği gibi kalır. Ama en azından üst sınıfa bu metodun imzası soyut metot olarak konur. Böylece müşteri ister regular ister preferred olsun, tür dönüşümü yapmadan doğrudan `chargeFor` çağrılabilir. Bu esneklik sağlar. Slaytta `createBill(Date)` de üst sınıfa çekilmiştir.

Template Method ile. Son özel durum: `chargeFor` metotlarında kodun bir kısmı ortak, bir kısmı farklıysa, Template Method kalıbı öğrenildiği gibi uygulanır. `chargeFor` içinde 4-5 ayrı iş

kalemi olsun. Alt sınıflarda değişmeyen, tamamen ortak olan kalemler üst sınıftaki `chargeFor` gövdesine kodlanır. Değişen kalemler ayrı soyut metotlar olarak çıkarılır; `chargeFor` işlemlerini bu soyut metotlar üzerinden yürütür ve alt sınıflar bu soyut metotları override eder.

> **Ders sonrası not:** Hoca önce üst sınıftaki `chargeFor` için "final metot olur" dedi, sonra "final olsa override edemeyiz" diyerek vazgeçti (02:04:21). Template Method kalıbında alt sınıfların override ettiği metotlar soyut adım metotlarıdır; iskeleti tutan şablon metodun (burada `chargeFor`) kendisi override edilmeyeceği için `final` tanımlanabilir. Büyük olasılıkla hocanın ilk söylediği kalıbın olağan kullanımına uygundur.

10. Long Method İçin Diğer Çözümler ve Decompose Conditional

10.1 Genel bakış

Long Method'u çözecek diğer refactoring eylemlerinin önce adları ve amaçları sayıldı:

- **Decompose Conditional**: "decompose" parçalarına ayırmak, "conditional" karar verme komutu (`if`) demektir. Hoca dersin başından beri söylenen bir şeyi hatırlattı: kendinizi `if`, `else`, `else`, `else` kodlarken ya da bir switch-case yapısı kurarken görürseniz, bu daha iyi bir tasarım yapmanız gerektiğinin en önemli alarm zillerinden biridir. Bunlar kodu uzatma eğilimindedir; çok sayıda döngü de kodu uzatır.
- **Replace Temp with Query**: geçici değişkenlerin yoğun kullanımı da kodu uzatma eğilimindedir. Geçici değişkenler (temp) yerine sorgu, yani metot çağrıları üzerinden hesap yapılır.
- **Replace Method with Method Object**: Extract Method, Pull Up Method, Decompose Conditional ya da Replace Temp with Query hâlâ tatmin edici bir çözüme götürmediyse kullanılır. Diğerlerinden epey karmaşıktır; hoca onu "ağır siklet" bir eylem olarak niteledi ve yalnız daha hafif, çevik eylemler işe yaramadığında yürütülmesi gerektiğini söyledi.

10.2 Refactoring: Decompose Conditional (karar verme komutunu parçalama)

Aşırı karmaşık `if` komutlarını sadeleştirmek anlaşılabilirliği artırır (ekrandan, 02:21:00):

```
/*** ÖNCE: ***/
if (date.before (SUMMER_START) || date.after(SUMMER_END))
    charge = quantity * _winterRate + _winterServiceCharge;
else charge = quantity * _summerRate;

/*** SONRA: ***/
if (notSummer(date))
    charge = winterCharge(quantity);
else charge = summerCharge (quantity);

private boolean notSummer(Date date) {
    return date.before (SUMMER_START) ||
        date.after(SUMMER_END);
}
private double summerCharge(int quantity) {
    return quantity * _summerRate;
}
private double winterCharge(int quantity) {
    return quantity * _winterRate + _winterServiceCharge;
}
```

Hoca önceki koşulu ilk bakışta anlamadığını açıkça söyledi ve anlamak için bir zaman çizgisi çizdi:

```
zaman ————————[SUMMER_START]==========[SUMMER_END]————————>
  boyanan:  ####                                    #########
            (yaz başlangıcından önce)          (yaz bitişinden sonra)
```

`SUMMER_START`'tan önceki ve `SUMMER_END`'den sonraki her yer boyandığında, yazın dışındaki her yerin boyandığı görülür. Bunu uzun uzun anlatmak yerine "yaz değilse" (`notSummer`) demek

çok daha iyi bir iştir; çok daha rahat anlaşılır.

Kodun geri kalanında: yaz değilse borç miktar × kış oranı + kış servis bedelidir; aksi hâlde miktar × yaz oranıdır. Uzun koşul anlaşılmasa bile belki metot gövdelerinden anlaşılabilir, ama gövdelerde de hata olabilir. Bu yüzden ne anlatılmak isteniyorsa onu az, öz ve doğru anlatmak daha yararlıdır. Ayrıca yaz oranı ve kış oranı diye iki ayrı üye ve bir kış hizmet bedeli olduğuna göre, yazın ve kışın yapılan hesaplama farklıdır; bunlar da kendi metotlarına çıkarılabilir.

Yapılanlar:

- Koşul `notSummer(Date)` adlı ayrı bir metoda çıkarıldı. Karar için tarihe ihtiyaç olduğundan tarih parametre olarak verilir. Metodun içindeki ifade pek değişmedi, ama `if`'in ne karar vermeye çalıştığı bir üst düzeyde çok daha iyi anlaşılır. Karar verirken hata varsa o metoda tıklanıp orada aranır. Önceki hâlde sorunun mevsim belirlemede mi yoksa hesaplanan miktarda mı olduğunu anlamak çok daha zordu.
- Yaz ve kış hesapları farklı olduğu için `summerCharge` ve `winterCharge` adlı iki ayrı metot oluşturuldu. İkisi de miktarı bilmek zorunda olduğu için `quantity` parametreyle iletilir.

Hoca, bunun karmaşıklığı halının altına süpürmek gibi görülebileceğini, ama buradaki karmaşıklığın işin doğasından geldiğini söyledi. Metodun adının `notSummer` olmasının nedeni, önceki hâldeki mantığı (yaz değilse kış bedeli) korumaktır. İstenirse mantık düz hâle çevrilip metot `isSummer` gibi adlandırılabilir ve `!` ile değillenerek kullanılabilir; bu bir sonraki iterasyonda yapılabilir. Hoca kendisinin böyle yapacağını, ama Fowler'ın böyle karar verdiğini, ikisinin de olabileceğini söyledi.

Özetle buradaki kod kusuru Long Method olarak algılanabilir: `if` koşulu çok karmaşık ve uzundur. Yapılan şey Decompose Conditional'dır: koşul toparlanmış, alt parçaları ayrı metotlara çıkarılmıştır. Alt parçaların ayrı metot olması zorunlu değildi; aynı metodun içinde de birkaç parçaya bölünebilirdi ve bu yine Decompose Conditional olurdu. Ama Extract Method bilindiği için onunla birlikte yürütmek en yararlısıdır.

11. Long Parameter List: Introduce Parameter Object ve Preserve Whole Object

11.1 Code smell: Long Parameter List (uzun parametre listesi)

`if` koşullarının karmaşıklığı bir sorunsa, metot parametre sayısının çok olması da bir sorundur. Slayttaki tanım (ekrandan, 02:30:00):

- Bir metodun parametre sayısının fazla olması, hatta bu parametrelerden bazılarının başka metotlarda da birlikte geçmesi durumudur.
  - Demek ki aslında bu parametreler birlikte bir varlığın durum bilgisinin bir kısmını (1) veya tamamını (2) oluşturmaktadır.
- Çözüme götürecek düzenleme:
  1. Introduce Parameter Object: metot parametrelerinin sayısı çok ise azaltmaya yönelik bir düzenlemedir.
  2. Preserve Whole Object: metot parametrelerinin sayısı çok ise azaltmaya yönelik diğer bir düzenlemedir.

Bir metot bir değil, iki değil, dört, beş, altı parametre almış olabilir. Üstelik diğer metotlara bakıldığında, bu parametrelerin bazı öbeklerinin başka metotlara da tekrar tekrar birlikte aktarıldığı görülebilir. Bu öbekler birbiriyle ilişkili verilerdir; bir araya geldiklerinde bir anlam taşıyorlarsa, bir varlık olarak kavramlaştırılabiliyorlarsa Introduce Parameter Object uygulanır. Parametreler aynı nesnenin birkaç getter'ından elde ediliyorsa Preserve Whole Object uygulanır: ayrı ayrı getter sonuçlarını vermek yerine nesnenin kendisi parametre olarak verilir.

11.2 Refactoring: Introduce Parameter Object (parametre nesnesi oluşturma)

Bir metot çok fazla parametre alıyorsa, uygun parametreler üye alan olarak yeni bir sınıfta toplanır ve o türden bir nesne parametre olarak kullanılır (ekrandan, 02:40:00):

```
ÖNCE:
Customer
```

```
SONRA:
Customer
```

```
amountInvoicedIn(start: Date, end: Date)     amountInvoicedIn(DateRange)
amountReceivedIn(start: Date, end: Date)     amountReceivedIn(DateRange)
amountOverdueIn(start: Date, end: Date)      amountOverdueIn(DateRange)
```

`Customer` sınıfındaki `amountXxx` metotlarının hepsi bir başlangıç ve bir bitiş tarihi alır. İki parametre çok uzun bir liste değildir; ama bu ikili çok yerde görülür. Bir başlangıç ve bir bitiş aslında bir zaman aralığından (period) söz eder. Bu yüzden sisteme `DateRange` adında, `start` ve `end` üye alanlarına sahip yeni bir sınıf eklenir ve metotlar bu sınıfın nesnelerini parametre olarak almaya başlar.

Slayttaki ek açıklama: bunu yalnız parametre listesini kısaltmak amacıyla yapmış olmazsınız.

- Oluşturulan `DateRange` sınıfı yalnız iki veriyi birleştiren bir kayıt yapısı olarak kalmaz.
- `isWithinRange(Date)` gibi uygun metotlar da içererek "decompose conditional" veya "extract method" düzenlemesi de yapılmış olunur.

Kodun geneline bakıldığında büyük olasılıkla başlangıç ve bitiş tarihlerini kullanan çok farklı işlemler de yapılmış olduğu görülecektir: bir şeyden önce mi, bir şeyden sonra mı, belirli bir tarih bir aralığın içinde mi gibi. Bu, kodun değişik yerlerinde tekrarlanan ya da parça parça dağılmış işlemler yeni sınıfa toplanabilir demektir. Böylece yalnız parametreler kısalmaz, kendi içinde uyumlu (cohesive) işlemler yeni bir sınıfta birikir; bir taşla iki kuş vurulur.

Tartışma: imza değişiyor. Hoca burada önemli bir tartışmayı kendisi açtı. Refactoring, metot imzalarını değiştirmeden düzenleme yapmaktı; ama bu eylemle imza değişti: önce iki ayrı parametre alınıyordu, şimdi tek ama farklı türden bir parametre alınıyor. Bu yüzden bu eylemin bir zayıf noktası vardır.

- Geriye doğru uyumluluk çok gerekliyse bu refactoring eylemi yapılmaz.
- Böyle bir yükümlülük yoksa yapılması çok daha iyidir; kazanımlar epey fazladır.

Hoca bu notu slayda bir açıklama olarak ekledi: "Metot imzasının değişmesi bir zayıflık olarak dikkati çekmelidir." Aynı şeyin sonraki slayt için de geçerli olduğunu yazdı (ekrandan, 02:40:00).

11.3 Refactoring: Preserve Whole Object (tüm nesneyi gönder)

Hoca terimin sözcük sözcük çevirisinin "tüm nesneyi koru" olduğunu, ama burada metot parametresi olarak göndermek söz konusu olduğundan "tüm nesneyi gönder" diye çevirdiğini söyledi. Bir nesnenin çeşitli üyelerini aynı metodun parametreleri olarak göndermek yerine nesnenin kendisi gönderilir (ekrandan, 02:37:10):

```
/*** ÖNCE: ***/
int low = daysTempRange().getLow();
int high = daysTempRange().getHigh();
withinPlan = plan.withinRange(low, high);

/*** SONRA: ***/
withinPlan = plan.withinRange(daysTempRange());
//withinRange metodu da gerektiği gibi değiştirilir.
```

Önceki hâlde `low` adlı geçici değişken, bir metodun döndürdüğü nesnenin `getLow()` metodu çağrılarak, `high` da aynı nesnenin `getHigh()` metodu çağrılarak elde edilir; sonra bu iki değer `plan` nesnesinin `withinRange` metoduna verilir. Sonraki hâlde `withinRange`'in imzası değiştirilir: `daysTempRange()` hangi türden bir nesne döndürüyorsa o türden tek bir parametre alır ve bu nesne doğrudan iletilir.

Zayıf yönler:

- Aynı tartışma burada da geçerlidir: metot imzası değişir.
- Ek olarak bağımlılık artabilir (ekrandan, 02:37:10). Parametre olarak gönderilen nesnenin sınıfı ile metodun sahibi olan sınıf arasında bir bağımlılık oluşur. Bu iki sınıf arasında daha önce hiçbir bağlaşım yoksa, birinin içinde diğerinin türünden bir nesnenin metodu kullanıldığında en azından UML'de kesikli çizgi ve okla gösterilen bağımlılık (kullanma) ilişkisi ortaya çıkar. Bağlaşım bir derece artmış olur.

> **Hocanın vurgusu:** Düşük bağlaşım tercih edilir ve bağlaşım daha önemli bir ölçüttür. İki sınıf arasında daha önce hiç ilişki yoksa ve bu bir sorun çıkaracaksa, bu ilişki kurulmaz; parametre listesi varsın uzun kalsın. İki sınıf arasında zaten bir ilişki varsa bu refactoring eylemi hiç çekinmeden yapılabilir.

Bu eylemle `low` ve `high` geçici değişkenleri de ortadan kalkmıştır; hoca bunun da bir kazanım olduğunu ve sıradaki eylemde (Replace Temp with Query) tartışılacağını söyledi.

12. Replace Temp with Query

12.1 Tanım ve örnek

Hoca bu eylemi "geçici değişken yerine erişim metodu kullanımı" diye çevirdi. Bir ifadeyi bir geçici değişkende saklayarak kullanmak yerine, ifadeden metot(lar) çıkarılır (ekrandan, 02:41:30):

```
/*** ÖNCE: ***/
double getPrice() {
    int basePrice = _quantity * _itemPrice;
    double discountFactor;
    if (basePrice > 1000) discountFactor = 0.95;
    else discountFactor = 0.98;
    return basePrice * discountFactor;
}

/*** SONRA: ***/
double getPrice() { return basePrice() * discountFactor(); }
private int basePrice() { return _quantity * _itemPrice; }
private double discountFactor() {
    if (basePrice() > 1000) return 0.95;
    else return 0.98;
}
```

Önceki hâlde `_quantity` ve `_itemPrice` üye alanlardır (alt çizgiden anlaşılır). Bunlar çarpılıp `basePrice` (taban fiyat) adlı geçici değişkende tutulur. Sonra taban fiyata göre değişen bir indirim oranı (`discountFactor`) belirlenir: taban fiyat 1000'den büyükse 0,95, yani %5 indirim; aksi hâlde 0,98, yani %2 indirim. Son komutta taban fiyat ile indirim oranı çarpılıp döndürülür.

`getPrice`'ın yaptığı üç ana iş vardır: taban fiyatı hesaplamak, indirim oranına karar vermek, ikisini çarpıp döndürmek. Sonraki hâlde bunlardan ikisi kendi metotlarına çıkarılır, üçüncüsü çarpma olarak kalır. `basePrice()` dışarıdan çağrılmaya ihtiyaç olmadığı için `private`'tır. `discountFactor()` içinde de geçici değişken kullanılmaz: taban fiyat `basePrice()` çağrılarak hesaplatılır ve 1000'den büyükse 0,95, değilse 0,98 döndürülür.

12.2 Geçici değişkenler neden kusur sayılır?

Hoca geçici değişkenlere neden "biraz alerji" duyulduğunu şöyle açıkladı: genelde optimizasyon için bir geçici değişken tanımlanır, sonra "bunu şu amaçla da kullanalım, bu amaçla da kullanalım" derken metot uzar. Ayrı, odaklanmış işlemler olarak kodlanabilecek şeylerin hepsi tek bir metotta toplanır; yani Long Method ortaya çıkar. Geçici değişken kullanımı Long Method kusurunu teşvik eden bir durumdur. Bu yüzden onların yerine metot çağrılarıyla hesaplayıp doğrudan vermek önerilir.

Slayttaki ek kazanım: bu eylem kopyala-yapıştır kodlamayı daha ortaya çıkmadan azaltır. Geçici değişkeni kullanan metotta yapılan işleme benzer bir işlem başka bir yerde kodlanmak istendiğinde, büyük olasılıkla "şeytan dürter" ve mevcut kod kopyalanıp yapıştırılır; Duplicated Code ortaya çıkar. Ama geçici değişkenlerle yapılan işlemler kendi sorgu (query) metotlarına toplanırsa, bir düzenleme gerektiğinde yalnız o sorgu metodu değiştirilir; kodun yapıştırıldığı bir sürü yerde değişiklik yapmak gerekmez. Bu önemli bir bakım avantajıdır.

12.3 Başarım tartışması

Sonraki hâlde bilgisayar fazladan bir işlem yapar: `getPrice` önce `basePrice()`'ı çağırır (çarpma yapılır), sonra `discountFactor()`'ı çağırır; o da içinden yeniden `basePrice()`'ı çağırır ve aynı çarpma bir kez daha yapılır. Ayrıca aynı metot iki kez çağrıldığı için çağrı ek yükü de vardır.

"Bilgisayar hesap yapmaktan gücenmez" denebilir; ama fazladan yapılan işlemler fazladan harcanan CPU komut çevrimleridir ve programın daha uzun sürmesine yol açabilir. Burada ise yalnız bir çarpma işlemi fazladır; ortaya çıkan gecikme bir insanın fark edebileceği bir süre değildir. Bu göz ardı edilebilir yüke karşılık kodun bakım masrafları azalır. Hoca bunu bir yatırım olarak düşünüp, bu yatırımın çok daha fazla geri dönüş (kâr) sağladığına karar verilmesini istedi. Çağrı ek yükünü de modern derleyiciler sıfıra ya da sıfıra yakın hâle getirmiştir; arka planda optimize kod üretilir. Bu yüzden bu refactoring eylemi rahatlıkla yapılabilir.

Uç durum: geçici değişken çok yerde kullanılıyorsa. Slayttaki tartışma (ekrandan, 02:50:00):

- Geçici değişken metot içerisinde birden fazla yerde kullanılıyorsa aynı işlem tekrar tekrar yapılacak.
- Bu da başarımı daha da fazla düşürecek.

```
void printRecipt() { /*** ÖNCE: ***/
    int totalAmount = 0, thisAmount;
    for( Item item : items ) {
        thisAmount = item.getCharge( );
        System.out.println(item.getName()+" "+thisAmount);
        totalAmount += thisAmount;
    } }

void printRecipt() { /*** SONRA: ***/
    int totalAmount = 0;
    for( Item item : items ) {
        System.out.println(item.getName()+" "+item.getCharge());
        totalAmount += item.getCharge( );
    } }
```

`totalAmount` tamamen kaldırılamaz, çünkü bu bir kümülatif toplam problemidir ve toplamı tutacak bir değişkene ihtiyaç vardır. Ama `thisAmount` kaldırılabilir. Hoca bunu IDE üzerinden anlattı: `thisAmount`'a tıklanınca her geçtiği yer vurgulanır. Değer bir noktada hesaplanıp atandıktan sonra ileride hiçbir noktada değiştirilmez. Ara hesaplama değişiklikleri barındırmayan, bir kez hesaplanıp her yerde değiştirilmeden kullanılan bu tür geçici değişkenler çok daha kolay kaldırılabilir. Refactor edilince `thisAmount` görülen her yere `item.getCharge()` sorgu çağrısı yazılır. Önceki hâlde altı çizili çağrı bir yerde, sonraki hâlde iki yerdedir. Bu bir for döngüsünün içinde olduğu için, öğe sayısı kadar fazladan çağrı yapılır ve başarımı daha ciddi düşürme potansiyeli vardır.

Örnekte her şey aynı bilgisayarın belleğinde çalışan kod üzerinde yapılıyor. Ama karmaşık bir bilgi sistemi düşünülürse (bankacılık ve telekom gibi alanlarda genelde ayrı işlemler için ayrı alt sistemler kullanılır ve aralarında haberleşme olur), sorgunun hesaplanması başka bir alt ağdaki başka bir sunucuya çağrı gerektiriyorsa, haberleşme trafiği ve gecikmesi katlanılabilir sınırın üstüne çıkabilir. O durumda bu refactoring eylemi yapılmaz.

Önerilen yol: profiler. Diğer durumlarda öneri şudur: korkmadan refactoring eylemini yapın. Profiler adı verilen yazılımlar, bir işlemin başından sonuna kadar ne kadar süre harcadığını ve bu sürenin ne kadarının hangi metotların çağrılmasında harcandığını ayrıntılı olarak gösterir.

- Refactoring eyleminin sisteme bir darboğaz getirdiği görülürse eylem geri alınır.
- Çoğu zaman aslında sistemin başka bir yerinde bir darboğaz vardır ve başarım sorunlarına o yol açıyordur. Önce çaba o darboğazı kaldırmaya yoğunlaştırılır.
- Hoca ters sıranın da daha doğru olabileceğini söyledi: önce bilinen daha sorunlu darboğazlar çözülür, oralar refactor edilir; sonra bu refactoring eylemi yürütülür. Yeni bir darboğaz getirmişse yapılmaz; ihmal edilebilir bir ek yük getirmişse yapılır, çünkü bakım masraflarını düşürür.

> **Hocanın vurgusu:** Replace Temp with Query, refactoring eylemleri içinde en ciddi çalışma zamanı (runtime) başarım kaybına yol açabilecek vakadır.

13. Replace Method with Method Object

Hoca bu eylemi "metot yerine yeni türden nesne kullanımı" diye çevirdi. Durum şudur: tasarımdan özel bir başarım sorunuyla karşılaşılmayacağı görülmüş ve geçici değişkenleri yok etmeye karar verilmiştir. Ama yerel geçici değişkenler öyle karmaşık, girift bir biçimde kullanılmıştır ki bir türlü Extract Method yapılamaz ya da nasıl yapılacağı ilk bakışta, hatta biraz çaba harcandıktan sonra bile anlaşılamaz.

Slayttaki özet (ekrandan, 02:57:00):

- Metot öyle uzun ve geçici yerel değişkenleri öyle "arap saçı" gibi kullanıyor ki "extract method" düzenlemesini kullanamıyorsunuz.
- O zaman yeni bir sınıf oluşturup bütün metodu o sınıfa taşıyın, yerel değişkenleri de o sınıfın üyeleri yapın.
- Ardından bildiğiniz düzenleme eylemlerini daha rahat yürütebilirsiniz.
- Yeni sınıfın adını verirken, taşınan metodun adına benzer bir ad verin.

Örnek:

```
/*** ÖNCE: ***/
class Order {
    double price() {
        double primaryBasePrice;
        double secondaryBasePrice;
        double tertiaryBasePrice;
        // long computation;
        ...
    }
}
```

```
SONRA:
  Order { price() }  - - - - >  PriceCalculator { primaryBasePrice;
                                                  secondaryBasePrice;
                                                  tertiaryBasePrice;
                                                  compute() }
  PriceCalculator ──1──> Order
  price() notu: return new PriceCalculator(this).compute()
```

`Order` sınıfının `price()` metodunda üç (belki daha fazla) geçici değişken tanımlanmış ve uzun bir hesaplama yapılıyor: değişkenlere önce atama yapılıyor, biraz kullanılıyor, sonra bir diğeri öbürü kadar artırılıp azaltılıyor, değişmiş hâliyle kullanılıyor; hepsi iç içe geçmiş. Bu durumda bunları ayrıntılı ayırmakla uğraşmak yerine en az çabayla en çok kazancın nasıl elde edileceğine bakılır: uzun metodun tamamı yeni oluşturulan bir sınıfa (`PriceCalculator`) taşınır ve metotta geçici değişken olarak tanımlanan değişkenler bu sınıfın üye alanları yapılır. Metodun adı aynı kalabilir ya da daha iyi bir adla (örnekte `compute`) değiştirilebilir.

`Order` sınıfında `price()` metodu eski adıyla kalır; yalnız gövdesi, yeni sınıfın metodunu çağırmaktan ibaret olur. Bu yüzden eylemin bir sakıncası yoktur: dışarıya verilen hizmet değişmemiştir, yalnız dışarıdan doğrudan kullanılmayan yeni bir sınıf oluşturulmuş ve işlemler onun üzerinden yapılır hâle gelmiştir. Karmaşık problem bir noktaya taşınıp odaklanmış olur; sonraki iterasyonlarda vakit kalırsa bu sınıf adım adım kendi içinde refactor edilir.

> **Ek bilgi (derste anlatılmadı):** Slayttaki şemada `PriceCalculator`'dan `Order`'a giden "1" çokluklu ilişki, yeni sınıfın kurucusuna verilen `this` ile kurulur. Böylece taşınan hesaplama, `Order`'ın gerekli alan ve metotlarına bu kaynak nesne üzerinden erişebilir.

14. Örnek Sorular

Soru 1 (ekrandan, 01:01:30)

`TestCustomer` sınıfındaki `testStatementWithMoviesLongRent` testi bir müşteriye The Matrix (normal film) 3 günlüğüne, Monsters, Inc. (çocuk filmi) 4 günlüğüne ve Surrogates (yeni film) 2 günlüğüne kiralatıyor. `statement()` metodunun ücret ve puan kurallarına göre beklenen makbuzdaki satır tutarları, toplam borç ve kazanılan puan nedir?

> **Çözüm (derste verildi; ekrandaki beklenen dizgiden):**
>
> - The Matrix: $2 + (3 - 2) \times 1{,}5 = 3{,}5$
> - Monsters, Inc.: $1{,}5 + (4 - 3) \times 1{,}5 = 3{,}0$
> - Surrogates: $2 \times 3 = 6{,}0$
> - Toplam borç: $12{,}5$. Puan: $1 + 1 + 1 + 1 = 4$ (yeni film 2 gün kiralandığı için 1 bonus).
> - Beklenen dizgi: `"Rental Record for ...\n\tThe Matrix\t3.5\n\tMonsters, Inc.\t3.0\n\tSurrogates\t6.0\nAmount owed is 12.5\nYou earned 4 frequent renter points"`.

Soru 2 (derste sözlü, 01:04:00)

Normal filmin günlük ek ücreti yanlışlıkla 1,5 yerine 2,5 kodlanırsa, yukarıdaki test nasıl bir sonuç verir ve JUnit hatayı nasıl gösterir?

> **Çözüm (derste verildi):** Test başarısız olur (kırmızı çubuk). The Matrix için $2 + (3 - 2) \times 2{,}5 = 4{,}5$ hesaplanır; JUnit beklenen “The Matrix 3.5” ile elde edilen “The Matrix 4.5” dizgilerinin ayrıştığı yeri köşeli parantezle gösterir. Matrix normal film olduğu için hatanın normal film ücretinin hesabında olduğu anlaşılır. Toplam borç da 13,5 olur (ara adım eklendi).

Soru 3 (derste sözlü, 01:02:38)

`testNewReleaseRentalBonus` testinde bir yeni film 2 günlüğüne, bir normal film 1 günlüğüne kiralanıyor. Kaç puan kazanılması beklenir?

> **Çözüm (derste verildi):** Yeni film: 1 puan + 2 gün kiralandığı için 1 bonus = 2 puan. Normal film: 1 puan. Toplam 3 puan.

Soru 4 (derste sözlü, 02:34:31)

Refactoring, dış arayüz (metot imzası) değişmeden iç yapıyı değiştirmek olarak tanımlandı. Introduce Parameter Object ve Preserve Whole Object bu tanımla nasıl bir gerilim içindedir ve hangi durumlarda yapılmamalıdır?

> **Çözüm (derste verildi):** İki eylem de metot imzasını değiştirir (iki ayrı parametre yerine tek, farklı türden bir parametre). Geriye doğru uyumluluk zorunluysa yapılmaz; değilse kazanımları büyüktür. Preserve Whole Object’te ek olarak, parametre nesnesinin sınıfı ile metodun sahibi sınıf arasında daha önce ilişki yoksa yeni bir bağımlılık doğar; bağlaşım artacağı için, bu sakıncalıysa eylem yapılmaz. İki sınıf zaten ilişkiliyse çekinmeden yapılabilir.

Soru 5 (ekrandan, 02:50:00)

`printRecipt()` metodunda `thisAmount` geçici değişkenine Replace Temp with Query uygulandığında başarım neden etkilenebilir? Ne zaman bu eylem yapılmamalıdır?

> **Çözüm (derste verildi):** `thisAmount` döngüde iki yerde kullanıldığı için, kaldırılınca `item.getCharge()` her öğe için iki kez çağrılır; yani öğe sayısı kadar fazladan çağrı yapılır. Hesap yerel ve hafifse bu ek yük önemsizdir. `getCharge()` başka bir alt ağdaki sunucuya çağrı gerektiriyorsa haberleşme gecikmesi kabul edilemez olabilir; bu durumda eylem yapılmaz. Genel öneri: eylemi yapın, profiler ile ölçün; darboğaz oluşturursa geri alın.

Dersin Toparlaması

Bu derste tasarım kalıplarından sonra dönemin ikinci büyük konusu olan refactoring ve kod kusurlarına giriş yapıldı. Refactoring, dış arayüzü koruyarak iç yapıyı ve iç kaliteyi iyileştirmektir; çevik biçimde, küçük adımlarla ve her adımdan sonra birim sınamaları çalıştırılarak yapılır. Bu yüzden önce JUnit ile birim sınaması (3.x'te `TestCase`, `setUp`, `assertXxx`, `suite`; 4.x'te annotation'lar) ve Fowler'ın video kaset kiralama örneği üzerinde yazılmış testler gösterildi. Ardından ilk kod kusurları ve onları düzelten refactoring eylemleri işlendi: Duplicated Code (Extract Method, Pull Up Method, Form Template Method), Long Method (Extract Method, Decompose Conditional, Replace Temp with Query, Replace Method with Method Object) ve Long Parameter List (Introduce Parameter Object, Preserve Whole Object). Her eylemin kazanımlarının yanında zayıf yönleri de tartışıldı: imza değişikliği, bağlaşımın artması ve başarım kaybı.

Hoca slaytların yaklaşık yarısının bittiğini, kalan kod kusurlarının ve refactoring eylemlerinin sonraki derste tamamlanacağını, ardından bütün eylemlerin kiralama örneği üzerinde adım adım uygulanacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.